



AI-HDL 2026: Design Phase 1 Documentation

NECRL at SFSU

Team Members :

Parsa Mirfasihi, Waylon Woo, Ethan Weldon, Vyshanvi Shekhar
Byrapatna

E-mail: smirfasihi@sfsu.edu, wwoo@sfsu.edu,
eweldon@sfsu.edu, vbyrapatnasomashekhar@sfsu.edu.

Phase title: Additional Module Synthesis

Deliverable: Synthesizable RTL, Verification Results and
Baseline Synthesis Report



Objective: (accelerator, peripheral, or instruction extension) into the provided base RISC-V core using an "AI-first" design methodology.

Design Choice: Advanced Encryption Standard (AES-128)

Overview

The Advanced Encryption standard (AES) is a symmetric-key bloc cipher standardized by the National Institute of Standards and Technology (NIST) and widely adopted embedded systems. AES operates on fixed-sized data blocks of 128 bits and supports key sizes of 128, 192, and 256 bits. In this work, AES-128 is implemented, which uses a 128-bit secret key and performs 10 rounds of encryption to transform a 128-bit plaintext block into a 128-bit ciphertext block.

AES-128 was selected due to its strong security guarantees, standardized specification, and favorable balance between computational complexity and hardware cost. It is commonly used in secure processors, embedded systems and communication protocols, making it suitable for hardware acceleration within RISC-V processor.

AES-128 Encryption Algorithm

AES encryption operates on an internal data structure known as the **state**, which represents the 128-bit input block as a 4×4 matrix of bytes. Encryption begins with an initial key addition and is followed by a sequence of transformation rounds that progressively diffuse and obscure the relationship between the plaintext, ciphertext, and secret key.

AES-128 consists of:

1. AddRoundKey

- ☐ The plaintext (128 bits) is arranged into a **4×4 byte matrix** called the **state**
- ☐ The state is XORed with **Round Key 0**

2. Main Rounds (Rounds 1-9)

Each of the 9 identical rounds consists of four transformations applied in this order:

1- SubBytes (Non-linear substitution)

- Each byte in the state is replaced using an S-box
- The S-box is:
 - Non-linear
 - Resistant to linear and differential cryptanalysis

2- ShiftRows (Row permutation)

- Rows of the state matrix are cyclically shifted:
- Row 0 → no shift
- Row 1 → left shift by 1
- Row 2 → left shift by 2
- Row 3 → left shift by 3

3- MixColumns (Column mixing)

- Each column is treated as a polynomial over $GF(2^8)$
- Multiplied by a fixed matrix:

$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix}$$

4- AddRoundKey

- XOR the state with the round key for that round

3. Final Round (Round 10)

Steps:

1. SubBytes
2. ShiftRows
3. AddRoundKey (Round Key 10)

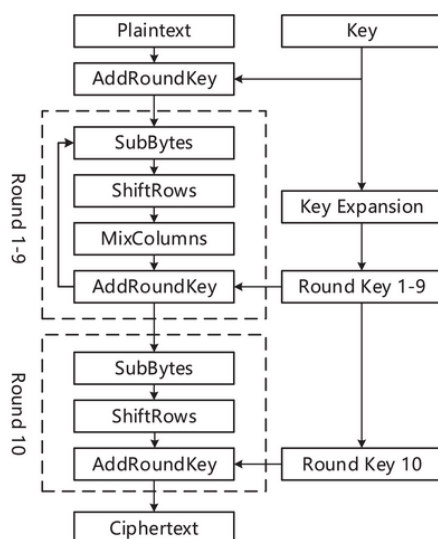


Figure 1. AES Round Transformations



Key Expansion

AES-128 employs a key expansion algorithm to derive a total of 11 round keys (one for the initial key addition and one for each of the 10 rounds). The key expansion process includes byte rotation, S-box substitution, exclusive-OR operations, and the use of round constants.

In this design, round keys are generated on-the-fly in synchronization with the encryption rounds, rather than being precomputed and stored. This approach reduces memory requirements and simplifies integration with an iterative encryption datapath.

Hardware Acceleration Benefits

Implementing AES as a dedicated hardware module provides several advantages over a purely software-based solution.

Performance

Software implementations of AES on embedded processors require a large number of instructions and memory accesses. A hardware accelerator executes the AES transformations directly in dedicated logic, significantly reducing execution time and providing deterministic latency.

Area and Energy Efficiency

An iterative hardware architecture reuses the same datapath across encryption rounds, resulting in a compact design with lower area and power consumption compared to fully unrolled implementations. This tradeoff is well suited for resource-constrained systems.

System Integration

Integrating AES as a memory-mapped peripheral allows encryption to be triggered and monitored by the processor using standard load/store operations. The use of status signaling and interrupts enables efficient synchronization between the processor and the encryption engine without excessive polling.

Why Integrate AES into a RISC-V Core?

Integrating AES as a hardware accelerator within a RISC-V processor enables secure computation while preserving software flexibility. RISC-V's open and modular architecture makes it well suited for custom peripherals and accelerators, allowing cryptographic operations to be offloaded from software to dedicated hardware.

By implementing AES as a memory-mapped peripheral, encryption can be initiated and monitored using standard load/store instructions, without modifying the processor pipeline or instruction set. This approach improves performance and energy efficiency while maintaining compatibility with existing RISC-V software flows.

AES Peripheral Architecture Overview

The AES peripheral was implemented using a modular architecture in which each subcomponent performs a distinct function within the encryption process. This organization improved design readability, simplified verification, and supported scalable integration within the RISC-V system. The AES peripheral consisted of the following major submodules:

- AES Memory Interface
- AES Control Unit
- AES Key Expansion Engine
- AES Encryption Engine
- AES S-box Lookup Table

These submodules were integrated and coordinated by a top-level peripheral module to form a complete hardware accelerator.

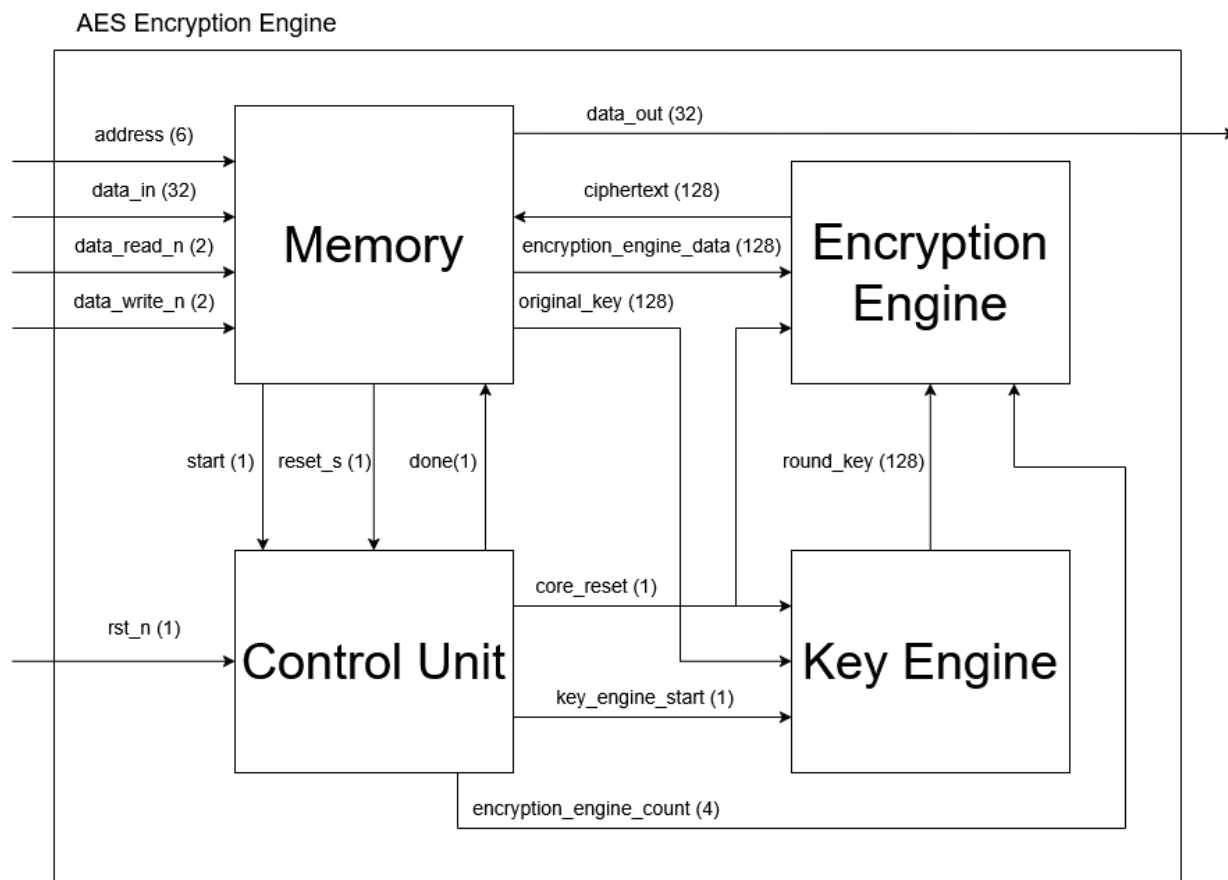


Figure 2. AI-HDL-AES-Engine



San Francisco State University - NECRL

AES Memory Module (AES_Memory)

The AES memory module was implemented to provide the interface between the RISC-V processor and the AES accelerator. It implemented a memory-mapped register file accessible through standard 32-bit read and write operations.

Responsibilities

The module stored:

- The 128-bit secret key as four 32-bit words
- The 128-bit plaintext input
- The 128-bit ciphertext output

It also provided control and status registers used to manage encryption execution.

Key Features

- A start control bit was used to initiate encryption
- A done status flag was asserted upon completion
- A key lock mechanism was included to prevent modification of the secret key after initialization
- Ciphertext data was automatically latched when encryption completed

This module isolated processor bus interactions from cryptographic logic, simplifying system integration and enabling independent verification of the register interface.

AES Control Unit (AES_control_unit)

The AES control unit was implemented to manage sequencing and synchronization of the encryption process. It functioned as a deterministic controller coordinating the key expansion and encryption datapaths.

Responsibilities

The control unit was responsible for:

- Generating a synchronized internal reset signal
- Maintaining a round counter to track encryption progress
- Initiating key expansion in coordination with encryption
- Asserting the done signal when encryption completed

Operation

Once encryption was initiated, the control unit advanced a round counter that:

- Identified the initial AddRoundKey operation
- Stepped through the nine main AES rounds
- Signaled the final encryption round
- Terminated the operation by asserting the completion signal

This counter-based control structure ensured correct ordering of AES operations and provided fixed and predictable encryption latency.

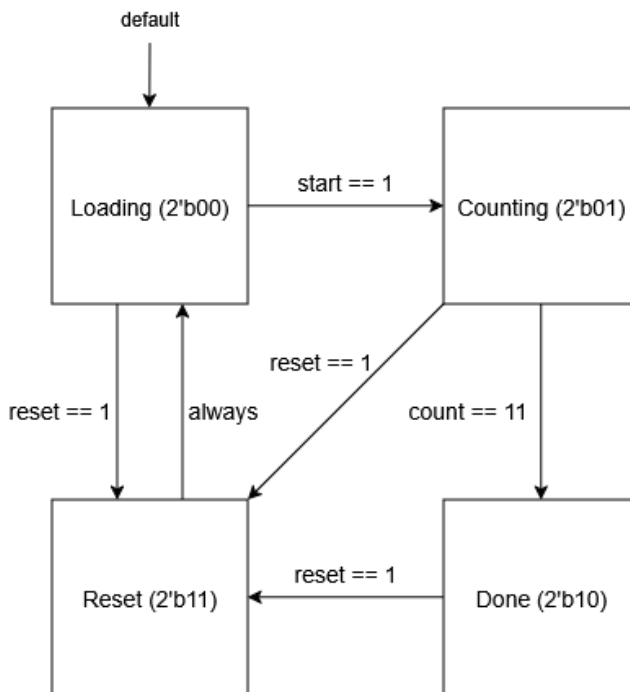


Figure 3. AES_Control_Unit

AES Key Expansion Engine (AES_key_engine)

The AES key expansion engine was implemented to generate the round keys required by the AES-128 algorithm. Rather than precomputing and storing all round keys, the design generated them on-the-fly in synchronization with the encryption rounds.

Responsibilities

The key expansion engine expanded the 128-bit input key into sequential round keys and applied the following AES key schedule operations:

- Word rotation
- Byte substitution using the AES S-box
- Round constant addition
- XOR-based word derivation

Design Rationale

On-the-fly key generation reduced memory requirements and integrated naturally with the iterative encryption engine. Each round key was produced precisely when needed, maintaining compliance with the AES specification while minimizing storage overhead.

Round Key Generation Steps

Given that each encryption round requires a different round key, round keys must be generated just before they are required. Our key generation engine/expander is designed in such a way that the round key for each round is generated one cycle before it's needed. The flowchart below illustrates the different states of the key engine. In summary, there are 11 states: one(PREP) for idling, R0 for outputting original key for initial addRoundKey step in the encryption engine and for precalculating round one round key. State R1 occurs on the next cycle and outputs round one round key and calculates round two round key. This occurs until we reach the final round(round ten). After that the key engine returns to PREP state to idle until control unit initiates the next calculation.

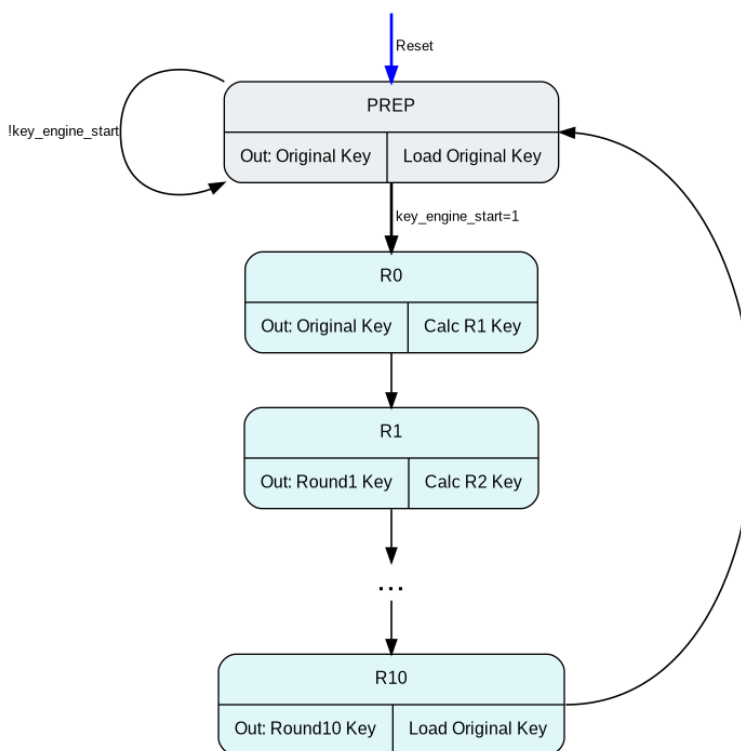


Figure 4. AES_Key_Engine

AES Encryption Engine (AES_encryption_engine)

The AES encryption engine was implemented to perform the core cryptographic transformations defined by the AES algorithm. It used an iterative datapath that executed one AES round per clock cycle.

Responsibilities

The encryption engine maintained an internal 128-bit AES state and performed the following AES transformations:

- SubBytes
- ShiftRows

- MixColumns
- AddRoundKey

Operation

- The initial round applied an AddRoundKey operation to the plaintext
- The main rounds applied all four AES transformations
- The final round omitted the MixColumns operation in accordance with the AES standard
- The final AES state was captured as the ciphertext output

This iterative approach balanced performance and hardware cost by reusing a single datapath across all encryption rounds.

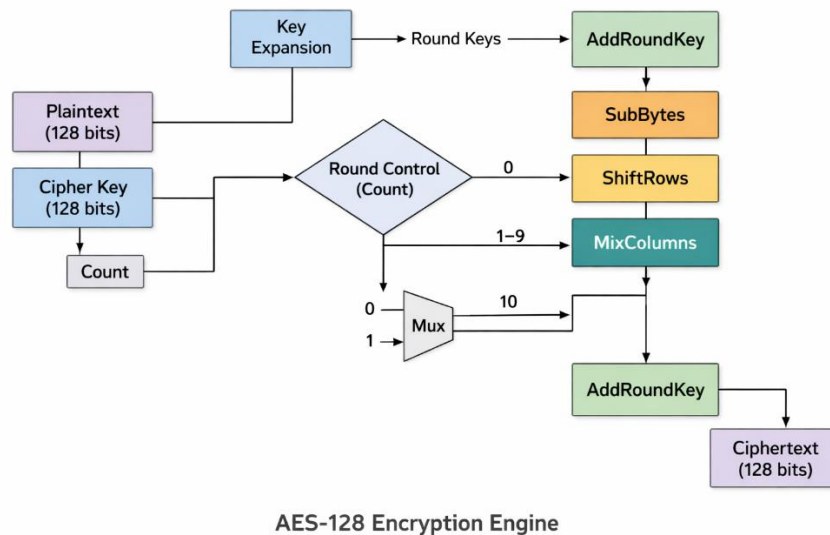


Figure 5. Encryption Engine flow

AES S-box Lookup (sbox_lookup)

The S-box module was implemented to provide the nonlinear byte substitution required by the AES algorithm. It was used by both the encryption engine and the key expansion engine.

Responsibilities

The module performed:

- Byte-wise substitution according to the AES S-box
- Consistent nonlinear transformations across encryption and key scheduling

Design Characteristics

The S-box was implemented as a purely combinational lookup table, enabling parallel substitution of multiple bytes per cycle without introducing additional control complexity.

Top-Level AES Peripheral (peripheral.v)

The top-level peripheral module was implemented to integrate all AES submodules and connect them to the RISC-V system bus, forming a complete hardware encryption accelerator.

Responsibilities

The top-level module connected:

- The AES memory interface to the processor
- Control signals to the AES control unit

It also coordinated data flow between:

- Memory registers
- The key expansion engine
- The encryption engine

Finally, the done signal was forwarded as an interrupt to the processor.

This integration allowed the AES accelerator to operate transparently alongside the processor pipeline and to be accessed through standard memory-mapped I/O.

Verification Strategy and Testbenches

The verification strategy for this AES accelerator employs a bottom-up, divide-and-conquer methodology, validating individual sub-modules before integrating them into the full system. This approach utilizes self-checking testbenches that compare Device Under Test (DUT) outputs against behavioral golden models implemented within the testbenches themselves. By isolating critical logic such as the Finite State Machine and the complex MixColumns transformation, we ensure unit-level correctness before moving to system-level validation using standard FIPS-197 test vectors.

```
(base) linux@linux-ThinkPad-T480s:~/AI-HDL/design_phase_1$ python3 run_tb.py

=== Select a Testbench to Run ===
[1] Top Level Integration (AES_peripheral_tb)
[2] Control Unit (AES_control_unit_tb)
[3] Encryption Engine (AES_encryption_engine_tb)
[4] Key Engine (AES_key_engine_tb)
[5] Memory Interface (AES_memory_tb)
[q] Quit
Enter choice: █
```

Figure 6. Initiating the Linux System for modules verification

The verification process begins with the Control Unit (AES_control_unit_tb.v), which validates the Finite State Machine (FSM) orchestrating the encryption flow. This testbench confirms that the internal counter transitions correctly from 0 to 11 or 12, aligning with the requisite AES rounds, and asserts the done signal at the precise clock cycle. It explicitly targets robustness by testing corner cases, such as asserting the start signal while an encryption is already in progress, initiating rapid back-to-back start pulses, and asserting reset mid-operation to ensure the FSM clears its state and recovers gracefully.

```
[INFO] Compiling Control Unit (AES_control_unit_tb)...
[INFO] Compilation Successful. Running Simulation...
-----
ASSERT PASS:                                done is 0 after reset
ASSERT PASS:                                enc count is 0 after reset
ASSERT PASS:                                key start is 0 after reset
ASSERT PASS: starts asserted within 10 cycles
195000: done asserted
ASSERT PASS:                                enc count is 0 when done asserted
ASSERT PASS:                                key start deasserted at done
ASSERT PASS:                                done is 1 at completion
ASSERT PASS:                                counter reached 11 before done
ASSERT PASS:                                done cleared after reset
ASSERT PASS:                                enc count cleared after reset
ASSERT PASS:                                key start cleared after reset
ASSERT PASS:                                done after repeated start during counting
ASSERT PASS:                                done remains 1 when start asserted during done
ASSERT PASS:                                done cleared by reset during counting
ASSERT PASS:                                enc count cleared by reset during counting
AES_control_unit_tb: ALL ASSERTS PASSED
AES128/testbenches/AES_control_unit_tb.v:170: $finish called at 985000 (1ps)
-----
```

Figure 7. Control Unit Verification Result

Following the control logic, the Encryption Engine (AES_encryption_engine_tb.v) verifies the datapath responsible for the mathematical transformations, including SubBytes, ShiftRows, and MixColumns. This testbench instantiates the AES_encryption_engine and compares its output against a full behavioral implementation of the AES logic functions included in the testbench file. A critical check involves verifying that the final round (Round 10) correctly bypasses the MixColumns step as per the AES specification, ensuring the final cipher output matches the behavioral model's calculation bit-for-bit.

```
[INFO] Compiling Encryption Engine (AES_encryption_engine_tb)...
[INFO] Compilation Successful. Running Simulation...
-----
PASS
AES128/testbenches/AES_encryption_engine_tb.v:266: $finish called at 116000 (1ps)
-----
```

Figure 8. Encryption Engine Verification Results

The Key Expansion module (AES_key_engine_tb.v) is verified by testing the on-the-fly key generation logic to ensure round keys are derived correctly from the initial user key. Unlike static vector tests, this testbench employs dynamic testing by running a loop with randomly generated keys and calculating the expected expansion using a "Software Engine" model implemented within the testbench. This method provides comprehensive coverage of the key expansion logic, including the RotWord and SubWord transformations, by verifying the generated round keys against the software model for every round.

```
[INFO] Compiling Key Engine (AES_key_engine_tb)...
[INFO] Compilation Successful. Running Simulation...
-----
STARTING DYNAMIC SELF-CHECKING TEST (Active High Reset)
-----

Test Case 0: Original Key = 12153524c0895e818484d609b1f05663
PREP State: Expected = 12153524c0895e818484d609b1f05663 | Actual = 12153524c0895e818484d609b1f05663
Round 0: Expected = 12153524c0895e818484d609b1f05663 | Actual = 12153524c0895e818484d609b1f05663
Round 1: Expected = 9fa4ceec5f2d906ddba946646a591007 | Actual = 9fa4ceec5f2d906ddba946646a591007
Round 2: Expected = 566e0bee09439b83d2eadde7b8b3cde0 | Actual = 566e0bee09439b83d2eadde7b8b3cde0
Round 3: Expected = 3fd3ea8236907101e47aace65cc96106 | Actual = 3fd3ea8236907101e47aace65cc96106
Round 4: Expected = ea3c85c8dcacf4c938d6582f641f3929 | Actual = ea3c85c8dcacf4c938d6582f641f3929
Round 5: Expected = 3a2e208be682d442de548c6dba4bb544 | Actual = 3a2e208be682d442de548c6dba4bb544
Round 6: Expected = a9fb3b7f4f79ef3d912d63502b66d614 | Actual = a9fb3b7f4f79ef3d912d63502b66d614
Round 7: Expected = da0dc18e95742eb304594de32f3f9bf7 | Actual = da0dc18e95742eb304594de32f3f9bf7
Round 8: Expected = 2f19a99bba6d8728be34cacb910b513c | Actual = 2f19a99bba6d8728be34cacb910b513c
Round 9: Expected = 1fc8421aa5a5c5321b910ff98a9a5ec5 | Actual = 1fc8421aa5a5c5321b910ff98a9a5ec5
Round 10: Expected = 9190e464343521562fa42eafa53e706a | Actual = 9190e464343521562fa42eafa53e706a
[PASS] Test Case 0 passed.

Test Case 1: Original Key = 06b97b0d46df998db2c2846589375212
PREP State: Expected = 06b97b0d46df998db2c2846589375212 | Actual = 06b97b0d46df998db2c2846589375212
Round 0: Expected = 06b97b0d46df998db2c2846589375212 | Actual = 06b97b0d46df998db2c2846589375212
Round 1: Expected = 9db9b2aadb662b2769a4af42e093fd50 | Actual = 9db9b2aadb662b2769a4af42e093fd50
Round 2: Expected = 43ede14b988bca6cf12f652e11bc987e | Actual = 43ede14b988bca6cf12f652e11bc987e
Round 3: Expected = 22ab12c9ba20d8a54b0fbd8b5ab325f5 | Actual = 22ab12c9ba20d8a54b0fbd8b5ab325f5
Round 4: Expected = 4794f477fdb42cd2b6bb9159ec08b4ac | Actual = 4794f477fdb42cd2b6bb9159ec08b4ac
Round 5: Expected = 671965b99aad496b2c16d832c01e6c9e | Actual = 671965b99aad496b2c16d832c01e6c9e
Round 6: Expected = 35496e03afe4276883f2ff5a43ec93c4 | Actual = 35496e03afe4276883f2ff5a43ec93c4
Round 7: Expected = bb957219147155719783aa2bd46f39ef | Actual = bb957219147155719783aa2bd46f39ef
Round 8: Expected = 9387ad5187f6f8201075520bc41a6be4 | Actual = 9387ad5187f6f8201075520bc41a6be4
Round 9: Expected = 2af8c44dad0e3c6dbd7b6e6679610582 | Actual = 2af8c44dad0e3c6dbd7b6e6679610582
Round 10: Expected = f393d7fb5e9deb96e3e685f09a878072 | Actual = f393d7fb5e9deb96e3e685f09a878072
[PASS] Test Case 1 passed.
```

Figure 9. Key Engine Verification Result

```

Round 3: Expected = 4af94d0eedeebd19510d364ef3694153 | Actual = 4af94d0eedeebd19510d364ef3694153
Round 4: Expected = bb7aa00356941d1a07992b54f4f06a07 | Actual = bb7aa00356941d1a07992b54f4f06a07
Round 5: Expected = 277865bc71ec78a6767553f2828539f5 | Actual = 277865bc71ec78a6767553f2828539f5
Round 6: Expected = 906a83afe186fb0997f3a8fb1576910e | Actual = 906a83afe186fb0997f3a8fb1576910e
Round 7: Expected = e8eb28f6096dd3ff9e9e7b048be8ea0a | Actual = e8eb28f6096dd3ff9e9e7b048be8ea0a
Round 8: Expected = f36c4fcbfa019c34649fe730ef770d3a | Actual = f36c4fcbfa019c34649fe730ef770d3a
Round 9: Expected = 1dbbcf14e7ba53208325b4106c52b92a | Actual = 1dbbcf14e7ba53208325b4106c52b92a
Round 10: Expected = 2bed2a44cc5779644f72cd742320745e | Actual = 2bed2a44cc5779644f72cd742320745e
[PASS] Test Case 7 passed.

Test Case 8: Original Key = c03b228010642120557845aacecccc9d
PREP State: Expected = c03b228010642120557845aacecccc9d | Actual = c03b228010642120557845aacecccc9d
Round 0: Expected = c03b228010642120557845aacecccc9d | Actual = c03b228010642120557845aacecccc9d
Round 1: Expected = 8a707c0b9a145d2bcf6c188101a0d41c | Actual = 8a707c0b9a145d2bcf6c188101a0d41c
Round 2: Expected = 6838e077f22cbd5c3d40a5dd3ce071c1 | Actual = 6838e077f22cbd5c3d40a5dd3ce071c1
Round 3: Expected = 8d9b989c7fb725c042f7801d7e17f1dc | Actual = 8d9b989c7fb725c042f7801d7e17f1dc
Round 4: Expected = 753a1e6f0a8d3baf487abbb2366d4a6e | Actual = 753a1e6f0a8d3baf487abbb2366d4a6e
Round 5: Expected = 59ec816a5361bac51b1b01772d764b19 | Actual = 59ec816a5361bac51b1b01772d764b19
Round 6: Expected = 415f55b2123eef770925ee002453a519 | Actual = 415f55b2123eef770925ee002453a519
Round 7: Expected = ec598184fe676ef3f74280f3d31125ea | Actual = ec598184fe676ef3f74280f3d31125ea
Round 8: Expected = ee6606e210016811e743e8e23452cd08 | Actual = ee6606e210016811e743e8e23452cd08
Round 9: Expected = f5db36fae5da5eeb0299b60936cb7b01 | Actual = f5db36fae5da5eeb0299b60936cb7b01
Round 10: Expected = dcfa4aff392014143bb9a21d0d72d91c | Actual = dcfa4aff392014143bb9a21d0d72d91c
[PASS] Test Case 8 passed.

Test Case 9: Original Key = cb203e968983b81386bc380da9a7d653
PREP State: Expected = cb203e968983b81386bc380da9a7d653 | Actual = cb203e968983b81386bc380da9a7d653
Round 0: Expected = cb203e968983b81386bc380da9a7d653 | Actual = cb203e968983b81386bc380da9a7d653
Round 1: Expected = 96d6d3451f556b5699e9535b304e8508 | Actual = 96d6d3451f556b5699e9535b304e8508
Round 2: Expected = bb41e341a41488173dfddb4c0db35e44 | Actual = bb41e341a41488173dfddb4c0db35e44
Round 3: Expected = d219f896760d70814bf0abcd4643f589 | Actual = d219f896760d70814bf0abcd4643f589
Round 4: Expected = c0ff5fccb6f22f4dfd028480bb417109 | Actual = c0ff5fccb6f22f4dfd028480bb417109
Round 5: Expected = 535c5e26e5ae716b18acf5eba3ed84e2 | Actual = 535c5e26e5ae716b18acf5eba3ed84e2
Round 6: Expected = 2603c62cc3adb747db0142ac78ecc64e | Actual = 2603c62cc3adb747db0142ac78ecc64e
Round 7: Expected = a8b7e9906b1a5ed7b01b1c7bc8f7da35 | Actual = a8b7e9906b1a5ed7b01b1c7bc8f7da35
Round 8: Expected = 40e07f782bfa21af9be13dd45316e7e1 | Actual = 40e07f782bfa21af9be13dd45316e7e1
Round 9: Expected = 1c748795378ea63aac6f9beeff797c0f | Actual = 1c748795378ea63aac6f9beeff797c0f
Round 10: Expected = 9c64f183abea57b90785cc57f8fcb058 | Actual = 9c64f183abea57b90785cc57f8fcb058
[PASS] Test Case 9 passed.

-----
ALL TESTS PASSED SUCCESSFULLY

```

Figure 10. Key Engine Verification – All test passed.

The Memory Interface (AES_memory_tb.v) acts as the CPU to validate the register map and the TinyQV-compatible bus interface. This testbench confirms that four 32-bit writes to the Data-In registers are correctly concatenated into the 128-bit encryption_engine_data signal and verifies that the done signal correctly captures the ciphertext into the output registers. It also tests security features by setting the key_lock bit in the Control Register and attempting to overwrite the key memory, confirming that the write is ignored and the original key remains intact.

```
Enter choice: 5

[INFO] Compiling Memory Interface (AES_memory_tb)...
[INFO] Compilation Successful. Running Simulation...
-----
59000: ASSERT PASS addr 0x2 => 0xaaaaaaaa
62000: ASSERT PASS addr 0x3 => 0xbbbbbbbbb
65000: ASSERT PASS addr 0x4 => 0xccccccccc
68000: ASSERT PASS addr 0x5 => 0xddddddddd
108000: ENGINE ASSERT PASS => 0x8765432112345678f0f0f0f0f0f0f0f
117000: start signal (from DUT) = 1
153000: ASSERT PASS addr 0xa => 0xd4d4d4d4
156000: ASSERT PASS addr 0xb => 0xc3c3c3c3
159000: ASSERT PASS addr 0xc => 0xb2b2b2b2
162000: ASSERT PASS addr 0xd => 0xa1a1a1a1
179000: ASSERT PASS addr 0x2 => 0xaaaaaaaa
202000: ASSERT PASS addr 0x2 => 0x00000000
205000: ASSERT PASS addr 0x3 => 0x00000000
208000: ASSERT PASS addr 0x6 => 0x00000000
211000: ASSERT PASS addr 0xa => 0x00000000
TESTBENCH DONE
AES128/testbenches/AES_memory_tb.v:149: $finish called at 222000 (1ps)
-----

=== Select a Testbench to Run ===
[1] Top Level Integration (AES_peripheral_tb)
[2] Control Unit (AES_control_unit_tb)
[3] Encryption Engine (AES_encryption_engine_tb)
[4] Key Engine (AES_key_engine_tb)
[5] Memory Interface (AES_memory_tb)
[q] Quit

Enter choice: █
```

Figure 11. Memory Engine Verification Result

Finally, the Top-Level Integration (AES_peripheral_tb.v) serves as the system-level acceptance test, treating the entire tqvp_neclr_spi peripheral as a black box interacting only through standard bus signals. It simulates a realistic usage driver sequence involving resetting the device, loading the key and plaintext, starting the engine, and waiting for the user interrupt. This testbench validates standard compliance using official FIPS-197 test vectors, confirming that the accelerator produces the cryptographically valid ciphertext 3925841d... from the known input key and plaintext, thereby verifying correct module interaction and endianness handling.

```
=== AES Peripheral Test Start ===
VCD info: dumpfile aes_test.vcd opened for output.
[INFO] Reset released.
[INFO] Loading Key...
[INFO] Loading Plaintext...
[INFO] Starting Encryption Engine...
[INFO] Waiting for Interrupt...
[INFO] Interrupt Received! Encryption Complete.
[INFO] Reading Ciphertext...

-----
Expected: 3925841d02dc09fdbc118597196a0b32
Got:      3925841d02dc09fdbc118597196a0b32
RESULT:   PASSED ✓
-----

[INFO] Testing Soft Reset...
[INFO] Soft Reset cleared control register successfully.
AES128/testbenches/AES_peripheral_tb.v:176: $finish called at 516000 (1ps)
```

Figure 12. Top-module verification.

All testbenches pass successfully, providing high confidence that the AES accelerator is functionally correct, compliant with the AES-128 standard, and robust against timing violations or illegal register access.

PPA Comparison: RISC-V CPU vs. Integrated System

The integration of the custom AES peripheral into the RISC-V baseline architecture results in a dramatic shift in the system's physical and operational profile. Moving from a standalone CPU to an integrated System-on-Chip (SoC) capable of cryptographic acceleration necessitates a substantial trade-off in silicon real estate and power consumption. This comparison highlights that while the baseline CPU is a minimal, lightweight core, the AES engine is a logic-dense accelerator that effectively dominates the final design metrics.

In terms of physical area, the difference is stark. The baseline CPU occupies a modest core area of approximately 16,493 square micrometers with a total cell count of 1,911. In contrast, the integrated system expands significantly, requiring a core area of 352,408 square micrometers and utilizing 48,355 cells. This represents a roughly 21-fold increase in area and a 25-fold increase in logic cells. Such exponential growth is characteristic of hardware AES implementations, which require extensive logic for S-Box substitution tables, 128-bit wide data paths, and state storage that dwarf the footprint of a compact CPU.

Power consumption metrics follow a similar upward trajectory, driven largely by the high switching activity of the encryption logic. The baseline design is extremely efficient, drawing a

San Francisco State University - NECRL

total typical power of roughly 1.25 nanowatts. The integrated system, however, consumes approximately 66.2 nanowatts—an increase of over 50 times. Specifically, switching power sees the most dramatic rise, jumping from negligible levels in the baseline to 0.0389 microwatts in the integrated design. This surge is consistent with the nature of cryptographic workloads, where complex combinatorial logic like MixColumns and continuous register updates across wide buses occur on every clock cycle.

Despite the added complexity, the system's timing performance remains robust. Both the baseline and the integrated designs comfortably meet the suggested clock frequency target of 67.57 MHz. The critical path delay did increase from 0.07 nanoseconds in the CPU-only configuration to 0.21 nanoseconds with the peripheral, reflecting the deeper logic levels introduced by the AES engine. However, this 0.21 nanosecond delay is still well within the timing margins for the target frequency, indicating that the design is not timing-constrained and retains significant headroom for potential frequency scaling.

Ultimately, this comparison underscores the cost of hardware acceleration. The integration of the AES peripheral transforms a lightweight, low-power risc-v core into a specialized cryptographic SoC, increasing the area footprint by over 2,000% and power consumption by over 5,000%. These increases are the necessary hardware investments required to offload intensive encryption tasks from the CPU, replacing thousands of slow software cycles with dedicated, high-speed hardware processing.

Metric	Baseline (CPU Only)	With AES Peripheral	Delta
Logic Cells	1,911	48,355	+2,430%
Core Area (μm^2)	16,493.32	352,407.99	+2,036%
Total Power (μW)	0.00125	0.0662	+5,196%
Switching Power (μW)	0.00004	0.0389	+9,625%
Internal Power (μW)	0.00085	0.0273	+3,111%
Critical Path Delay (ns)	0.07	0.21	+200%
Target Frequency (MHz)	67.57	67.57	0%
Target Frequency (MHz)	67.57	67.57	0%

Figure 13. PPA Results.

Please note that all other deliverables (Full RTL, Testbenches, LLM Prompt Logs, Diagrams and PPA metrics) are located in the DP1-Submission folder on GitHub.